

Track B: Python for data science

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

- Track B: Python for data science2**
 - B1 Project setup 3
 - B1-L1 One pyproject.toml and a src layout 3
 - B1-L2 One virtual environment per project, and python -m pip 7
 - B1-L3 ruff and mypy in daily work 10
 - B2 Code design 15
 - B2-L1 Pure functions, side effects at the edges 15
 - B2-L2 Dataclasses as configuration objects 19
 - B2-L3 When a class earns its place 23
 - B3 Clean pandas 28
 - B3-L1 Dtypes in pandas 3, nullable and string 28
 - B3-L2 Vectorise instead of apply 31
 - B3-L3 Method chaining and Copy-on-Write 36
 - Track B best-practice checklist 41
- Appendix: solutions and answer keys42**

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B1 Project setup

Roadmap: Prerequisite for all months. pyproject.toml, virtual environments, ruff, mypy and folder layout.

B1-L1 One pyproject.toml and a src layout

Concept

A data science project starts as a notebook and a CSV. By month three it has helper modules, tests, a model, and three people asking how to run it. The fix is structure from day one.

pyproject.toml is the one configuration file for a modern Python project. It holds:

- [project]: name, version, requires-python, and runtime dependencies.
- [dependency-groups] (PEP 735): named groups for development only, such as test or lint. They never ship with the package. Install one with `pip install --group test` (pip 25.1 and later).
- [tool.*] tables for tools: [tool.ruff], [tool.mypy], [tool.pytest.ini_options].

One file replaces setup.py, requirements.txt, setup.cfg, .flake8 and pytest.ini.

The src layout puts importable code under `src/`:

```
1 reach_model/
2 └─ pyproject.toml
3 └─ src/reach_model/      # importable package
4   └─ __init__.py
5   └─ features.py
6   └─ train.py
7 └─ tests/                # pytest, imports reach_model
8 └─ notebooks/           # exploration; imports reach_model, holds no
    logic
9 └─ data/                # small samples only; real data stays out of
    Git
```

With `src/`, Python can only import your package after you install it (`pip install -e .`). Tests then run against the installed package, exactly as a user or a job would. Without it, tests silently import whatever sits in the working directory, and "works on my machine" follows.

Notebooks explore and present. Functions that matter move into `src/`, where tests and type checks reach them. A notebook that defines the feature pipeline inline cannot be tested or reused by the training job.

Pin `requires-python` to the versions you test, for example `">=3.11"`, so pip refuses an interpreter you never tried.

Why it matters

A team trains a churn model from `train.ipynb`, which defines `build_features()` in a cell. The scoring job copies the function into `score.py`. A month later someone fixes a leakage bug in the notebook version only. Training and scoring now compute features differently, and live accuracy drops 9 points while offline metrics look fine.

With `build_features()` in `src/churn/features.py`, imported by both the notebook and the job, one fix reaches both, and one test protects it.

Worked example: weak code

1	churn/	
❶	├─ train.ipynb	# defines build_features() in cell 7
❷	├─ score.py	# copy of build_features(), edited later
4	├─ utils.py	
❸	├─ requirements.txt	# pandas, scikit-learn (no versions)
6	├─ setup.cfg	# flake8 settings
❹	└─ data.csv	# 2 GB client extract, committed

1. **Line 2.** Logic trapped in a notebook: untestable, and copied instead of imported.
2. **Line 3.** Two versions of one function drift apart.
3. **Line 5.** No pins: every install resolves different versions.
4. **Line 7.** Client data in Git: a size problem and a data-protection problem.

Worked example: strong code

1	churn/	
❶	├─ pyproject.toml	# project, dependency groups, ruff, mypy, pytest
3	├─ src/churn/	
4	├─ └─ __init__.py	
❷	├─ └─ features.py	# build_features(), used by training and scoring
6	├─ └─ train.py	
7	├─ └─ score.py	
8	├─ tests/	
❸	├─ └─ test_features.py	
10	├─ notebooks/	
11	├─ └─ 01_explore.ipynb	# imports churn.features

```
12 └─ data/
    4 └─ sample_200_rows.csv
```

1. **Line 2.** One file configures the project and every tool.
2. **Line 5.** One definition, imported everywhere it is needed.
3. **Line 9.** Tests import the installed package, as the jobs do.
4. **Line 13.** A small, anonymised sample for tests. Real data stays in governed storage.

Logic lives once, in an installable package. Notebooks, jobs and tests import it.

Recap card

- **What goes in `pyproject.toml`?** `[project]` with dependencies, `[dependency-groups]` for development tools, and `[tool.*]` settings for `ruff`, `mypy` and `pytest`.
- **Install a dependency group** `pip install --group dev (pip 25.1 and later)`.
- **Why the `src` layout?** Only the installed package is importable, so tests run against it exactly as jobs do.
- **Where does shared logic live?** In the package under `src/`, imported by notebooks, jobs and tests. Never copied.

Practice

Exercise 1. Multiple choice (level 1)

Where do you declare `pytest` and `ruff` so they install for developers but never ship with the package?

- a) `[project]` dependencies
- b) `[dependency-groups]`
- c) `[tool.pytest.ini_options]`
- d) A comment in `README.md`

Why?

- r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.
- r2) Dependency groups are only for test tools.

Exercise 2. Fill the gap (level 2)

Complete the `pyproject.toml` so runtime needs and developer tools are separate.

```
1 [project]
```

```

2 name = "churn"
3 requires-python = ">=3.11"
4 ____ (1) = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]
5
6 [____ (2)]
7 dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]

```

Exercise 3. Write the function (level 3)

Write `unpinned(pyproject_text)`. Parse the TOML with the standard library and return, sorted, every entry in `[project].dependencies` that has no version constraint at all (no `=`, `<`, `>`, `~` or `!`). Return an empty list when there is no dependencies key.

Starter code:

```

1 import tomllib
2
3
4 def unpinned(pyproject_text: str) -> list[str]:
5     ...

```

Exercise 4. Multiple choice (level 4)

With a src layout, a colleague runs `pytest` straight after cloning and gets `ModuleNotFoundError: No module named 'churn'`. What is the right fix?

- a) Add `sys.path.insert(0, 'src')` at the top of every test file.
- b) Run `pip install -e .` in the project's virtual environment.
- c) Move the package out of `src/` so imports work from the root.
- d) Copy the package into `tests/`.

Why?

- r1) The src layout is designed so that only the installed package is importable, which makes tests use it like any user would.
- r2) pytest cannot find packages in folders.

Exercise 5. Architecture design (level 5)

Place each piece of a modelling project.

Options: `src/<package>/`, A notebook cell, `tests/`, Outside the repository.

1. `build_features()`, used by training and by the nightly scoring job
2. Plots comparing feature distributions for a one-off question
3. A check that `build_features()` gives the right output for 5 hand-made rows
4. The full client data extract

Sources

- PEP 735 defines [dependency-groups]; pip 25.1 (April 2025) added --group. (search-only: <https://peps.python.org/pep-0735/> and <https://ichard26.github.io/blog/2025/04/whats-new-in-pip-25.1/>)
- pip 26.2.1 lists --group in `pip install --help`; tomllib is in the standard library from Python 3.11. (verified: Checked in this repository's build environment.)

B1-L2 One virtual environment per project, and python -m pip

Concept

A **virtual environment** is a folder with its own Python interpreter link and its own `site-packages`. Packages you install there do not touch the system Python or any other project.

```
1 python -m venv .venv
2 source .venv/bin/activate      # Windows: .venv\Scripts\activate
3 python -m pip install --upgrade pip
4 python -m pip install -e . --group dev
```

Rules that prevent most "it worked yesterday" problems:

- **One environment per project.** Two projects that share an environment share their version conflicts. Upgrading pandas for one breaks the other.
- **Never install into the system Python.** Operating systems and other tools depend on it.
- **Use** `python -m pip`, **not bare** `pip`. Bare `pip` runs whichever `pip` comes first on your PATH, which may belong to a different interpreter. `python -m pip` always installs into the interpreter you are about to run.
- **Keep** `.venv/` **out of Git.** It is large, machine-specific and rebuildable. Commit `pyproject.toml` and a lock file instead (lesson B6-L3).

In code, `sys.prefix != sys.base_prefix` is True inside a virtual environment. A script that must not run against the system Python can check it and stop.

Sometimes one project needs **two** environments, because two dependencies cannot live together. A real case from this course's own build: PySpark 4.0.4 fails to import its pandas integration with pandas 3.x, so the Spark test harness runs in its own environment pinned to pandas 2.x, while the rest uses pandas 3. Separate environments, each fully pinned, beat one environment held together by guesswork.

Tools such as `uv` create and sync environments much faster, and `conda` manages non-Python libraries. The principles are the same.

Why it matters

An analyst runs `pip install statsmodels` in a terminal, then `python analysis.py` in the same terminal, and gets `ModuleNotFoundError`. `pip` belonged to one Python, and `python` to another. They try `sudo pip install`, which upgrades NumPy in the system Python and breaks a command-line tool the operating system uses. Two hours are gone.

`python -m venv .venv`, activation, and `python -m pip install` make the interpreter that installs and the interpreter that runs the same one.

Worked example: weak code

```
❶ $ pip install pandas scikit-learn
❷ $ sudo pip install --upgrade numpy
3 $ python train.py
❸ ModuleNotFoundError: No module named 'sklearn'
❹ $ git add .venv/ data/ train.py
```

1. **Line 1.** Bare pip, no environment: installs into whichever Python owns this pip.
2. **Line 2.** Changes the system Python that the operating system relies on.
3. **Line 4.** The python that runs is not the one pip installed into.
4. **Line 5.** Commits a machine-specific environment folder and data.

Worked example: strong code

```
❶ $ python -m venv .venv
2 $ source .venv/bin/activate
3 (.venv) $ python -m pip install --upgrade pip
❷ (.venv) $ python -m pip install -e . --group dev
5 (.venv) $ python -c "import sys; print(sys.prefix != sys.base_prefix)"
❸ True
7 (.venv) $ python train.py
```

1. **Line 1.** A project-local environment, listed in `.gitignore`.
2. **Line 4.** The same interpreter installs and runs. Dependencies come from `pyproject.toml`.
3. **Line 6.** Proof that the active interpreter is the virtual environment.

One environment per project, created from `pyproject.toml`, with `python -m pip` so installing and running use the same interpreter.

Recap card

- **Create and use a project environment** `python -m venv .venv`, activate it, then `python -m pip install -e . --group dev`.

- **Why python -m pip?** It installs into the interpreter that python runs. Bare pip may belong to another Python.
- **Detect a virtual environment in code** `sys.prefix != sys.base_prefix`.
- **Two dependencies that cannot coexist** Two pinned environments and two dependency groups, each tested in CI.

Practice

Exercise 1. Multiple choice (level 1)

Why prefer `python -m pip install x` over `pip install x`?

- a) It is faster.
- b) It installs into the interpreter that `python` runs, whatever is first on `PATH`.
- c) It installs packages globally for all users.
- d) It skips dependency resolution.

Why?

- r1) `python -m pip` runs pip as a module of that exact interpreter.
- r2) Bare pip is deprecated.

Exercise 2. Predict the output (level 2)

You run this with `.venv/bin/python`, a virtual environment created by `python -m venv .venv`. What does it print?

```
1 import sys
2
3 print(sys.prefix != sys.base_prefix)
```

Exercise 3. Fill the gap (level 2)

Create the environment, then install the project in editable mode with its dev group.

```
1 python -m ____ (1) .venv
2 source .venv/bin/activate
3 python -m pip install ____ (2) . --group dev
```

Exercise 4. Write the function (level 3)

Write `require_virtualenv()`. It raises `RuntimeError` with the message "activate the project's virtual environment first" when the running interpreter is not inside a virtual environment, and returns `None` otherwise. Read `sys.prefix` and `sys.base_prefix` when the function is called, not at import time.

Starter code:

```
1 import sys
2
3
4 def require_virtualenv() -> None:
5     ...
```

Exercise 5. Architecture design (level 5)

Your project needs pandas 3.0 for analysis, and a Spark test harness on PySpark 4.0.4, which fails to import its pandas integration with pandas 3.x. Choose the set-up.

Options: Two environments: dev (pandas 3) and spark (pandas 2), each pinned, One environment with the latest of everything, One environment, everything downgraded to pandas 2.

1. Environments
2. Where the difference is recorded
3. CI

Sources

- `sys.prefix` differs from `sys.base_prefix` inside a `venv`. (verified: Python `venv` documentation; checked in this repository's virtual environments.)
- PySpark 4.0.4 fails to import `assertDataFrameEqual` with pandas 3.x. (verified: Reproduced in this repository (VERSIONS.md).)

B1-L3 ruff and mypy in daily work

Concept

Two tools catch most mistakes before a test runs.

ruff lints and formats. On a ten-line sample it reported three real problems:

```
1 m.py:1:1: I001 [*] Import block is un-sorted or un-formatted
2 m.py:1:8: F401 [*] `os` imported but unused
3 m.py:9:30: B905 `zip()` without an explicit `strict=` parameter
```

B905 matters for data work: `zip(flags, weights)` silently stops at the shorter list, so a missing weight drops a respondent without an error. `zip(..., strict=True)` raises instead.

mypy checks types. On the same file, strict mode found:

```
1 m.py:14: error: Unsupported operand types for * ("None" and "int")
  [operator]
2 m.py:17: error: Function is missing a type annotation [no-untyped-def]
```

Line 14 multiplies a `float | None` by 100. When the total weight is zero, the function returns `None` and the report crashes. `mypy` found the zero-weight bug without any test data.

Daily habits:

- Configure both once in `pyproject.toml` (`[tool.ruff]` with the rule sets you want, `[tool.mypy]` with `strict = true`).
- Run `ruff check --fix .` and `ruff format .` before each commit. `--fix` applies only safe fixes.
- Run `mypy` on `src/` and `tests/`. Fix errors by handling the case, not by adding `# type: ignore`.
- Run the same commands in CI (lesson A10-L3), so the rules are not optional.

Libraries without type information show `import-untyped` or `missing-stub` notes. Install the stub package if one exists, or add a targeted `[[tool.mypy.overrides]]` with `ignore_missing_imports = true` for that module only.

Why it matters

A reporting script formats a weighted share as `f"{share * 100:.1f}%"`. For eleven months every subgroup has respondents. Then a new region with no fieldwork yet appears in the sample frame, its total weight is zero, `weighted_share` returns `None`, and the monthly client pack fails to build on deadline day.

`mypy` flags `None * 100` the first time it runs on the file. The fix is two lines, written months before the failure.

Worked example: weak code

```
1 import os
2 import pandas as pd
3
4
5 def weighted_share(flags, weights):
6     total = sum(weights)
7     if total == 0:
8         return None
9
10    return sum(w for f, w in zip(flags, weights) if f == 1) / total
11
12 def report(flags, weights):
13     return f"{weighted_share(flags, weights) * 100:.1f}%"
```

1. **Line 1.** F401: unused import.

2. **Line 5.** No hints: mypy cannot see that the function can return None.
3. **Line 9.** B905: zip silently truncates to the shorter list.
4. **Line 13.** Crashes with TypeError when the share is None.

Worked example: strong code

```
❶ def weighted_share(flags: list[int], weights: list[float]) -> float |  
    None:  
    2     """Weighted share of flagged rows, or None when the total weight is  
        0."""  
    3     total = sum(weights)  
    4     if total == 0:  
    5         return None  
❷     return sum(w for f, w in zip(flags, weights, strict=True) if f ==  
        1) / total  
    7  
    8  
    9 def report(flags: list[int], weights: list[float]) -> str:  
    10     """Share as a percentage string, or 'n/a' when undefined."""  
    11     share = weighted_share(flags, weights)  
❸     if share is None:  
    13         return "n/a"  
    14     return f"{share * 100:.1f}%"
```

1. **Line 1.** The return type admits None, so mypy makes every caller handle it.
2. **Line 6.** Unequal lengths raise ValueError instead of dropping rows.
3. **Line 12.** The case mypy flagged, handled on purpose.

ruff and mypy found an unused import, a silent truncation and a crash on zero weight, before anyone wrote a test.

Recap card

- **Why does ruff flag zip() without strict= (B905)?** zip silently truncates to the shorter input. strict=True raises ValueError instead.
- **mypy says None * int is unsupported. The fix?** Handle the None case in code. Never # type: ignore it away.
- **Daily commands before a commit** ruff check --fix ., ruff format ., mypy.
- **Missing stubs for one library** Install types-<lib> if it exists, otherwise a per-module mypy override. Never global.

Practice

Exercise 1. Predict the output (level 1)

What does this print?

```
1 flags = [1, 0, 1]  
2 weights = [2.0, 1.0]
```

```
3 print(list(zip(flags, weights)))
```

Exercise 2. Multiple choice (level 2)

mypy reports `Unsupported operand types for * ("None" and "int")` on `share * 100`. What is the right fix?

- a) Add `# type: ignore` to the line.
- b) Handle the `None` case before the multiplication.
- c) Change the return type to `float` and keep returning `None`.
- d) Turn off strict mode.

Why?

- r1) mypy found a real path where the value is `None`. Handling it removes the crash.
- r2) mypy errors are style issues.

Exercise 3. Fill the gap (level 2)

Complete the pre-commit routine: apply safe lint fixes, format, then type-check.

```
1 ruff check ____ (1) .
2 ruff ____ (2) .
3 ____ (3)
```

Exercise 4. Refactor (level 4)

Make this pass `ruff` (no B905) and `mypy --strict`: add type hints, keep the `None` result for zero weight, make unequal lengths raise, and make `report` return `"n/a"` instead of crashing.

Starter code:

```
1 def weighted_share(flags, weights):
2     total = sum(weights)
3     if total == 0:
4         return None
5     return sum(w for f, w in zip(flags, weights) if f == 1) / total
6
7
8 def report(flags, weights):
9     return f"{weighted_share(flags, weights) * 100:.1f}%"
```

Exercise 5. Architecture design (level 5)

mypy reports `Skipping analyzing "shapely": module is installed, but missing library stubs or py.typed marker`. Choose the response for each case.

Options: Install the stubs in the dev group, Add `[[tool.mypy.overrides]]` with `ignore_missing_imports` for that module, Set `ignore_missing_imports = true` globally.

1. A stub package, `types-shapely`, exists on PyPI
2. No stubs exist for an internal library
3. A colleague proposes silencing all such messages for every library

Sources

- ruff reports `I001`, `F401` and `B905` and mypy `--strict` reports `[operator]` and `[no-untyped-def]` on the sample shown. (verified: ruff 0.16.9 and mypy 2.3.1 run in this repository.)
- `zip(strict=True)` raises `ValueError` on unequal lengths (Python 3.10+). (verified: Python standard library; run in Python 3.11.)

B2 Code design

Roadmap: Prerequisite for all months. Pure functions, dataclasses, when to use classes, configuration objects.

B2-L1 Pure functions, side effects at the edges

Concept

A **pure function** returns a value that depends only on its arguments, and changes nothing outside itself: no file writes, no global variables, no mutation of the inputs, no printing, no clock or random calls it does not receive.

```
1 def weighted_mean(values: list[float], weights: list[float]) -> float:
2     return sum(v * w for v, w in zip(values, weights, strict=True)) /
    sum(weights)
```

Pure functions are easy to test (inputs in, answer out), safe to reuse, and safe to run twice. Most analysis logic can be pure: cleaning, feature building, statistics, scoring.

Some work is not pure by nature: reading files, calling APIs, writing tables, logging, reading the clock. Push that work to the **edges** of the program, into a thin layer that gathers inputs, calls the pure core, and writes results.

```
1 read (impure) -> transform, compute (pure) -> write (impure)
```

Three side effects hide easily in data code:

- **Mutating an argument.** `df["x"] = ...` or `rows.append(...)` inside a function changes the caller's object. The caller did not ask for that. Return a new object instead: `df.assign(x=...)`, or build a new list.
- **Reading globals.** A function that reads a module-level `THRESHOLD` or `TODAY` gives different answers after someone changes it. Pass it in.
- **Hidden time and randomness.** `datetime.now()` or `np.random.rand()` inside a function makes its output untestable. Pass a date, or a `numpy.random.Generator`.

In pandas 3, Copy-on-Write is always on, so many operations already return new objects. Assigning a column with `df[col] = ...` still changes `df` itself, which is exactly the mutation to keep out of shared functions.

Why it matters

A feature function adds `age_group` to the DataFrame it receives, in place. The training script calls it, then passes the same DataFrame to a stratified split on `age_group`. Later, someone reorders the calls so the split runs first. The split fails with a `KeyError` on one laptop and works on another, depending on which notebook cells ran before.

A pure `add_age_group(df) -> DataFrame` that returns a new frame gives the same result in every order, and its test needs three rows.

Worked example: weak code

```
1 import datetime
2
3 THRESHOLD = 0.5
4
5
6 def score_respondents(df):
7     df["age"] = datetime.date.today().year - df["birth_year"]
8     df["high_reach"] = df["reach"] > THRESHOLD
9     df.to_csv("scored.csv", index=False)
10    print("scored", len(df))
11    return df
```

1. **Line 7.** Mutates the caller's frame and reads the clock: the answer changes on 1 January.
2. **Line 8.** Reads a module global: change `THRESHOLD` anywhere and every result changes.
3. **Line 9.** Writes a file from inside the computation.
4. **Line 10.** Output in the middle of logic, not in the program's edge.

Worked example: strong code

```
1 import pandas as pd
2
3
4 def score_respondents(df: pd.DataFrame, as_of_year: int, threshold:
5     float) -> pd.DataFrame:
6     """Add age and high_reach. Returns a new frame; the input is
7     unchanged."""
8     return df.assign(
9         age=as_of_year - df["birth_year"],
10        high_reach=df["reach"] > threshold,
11    )
12
13 def main(input_path: str, output_path: str, as_of_year: int, threshold:
14     float) -> None:
15     """The edge: read, call the pure core, write."""
```

```

14     scored = score_respondents(pd.read_csv(input_path), as_of_year,
    threshold)
15     scored.to_csv(output_path, index=False)

```

1. **Line 4.** Everything the result depends on is a parameter.
2. **Line 6.** assign returns a new frame. The caller's data is not touched.
3. **Line 12.** Reading, writing and choosing the year happen here, at the edge.

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

Recap card

- **What makes a function pure?** Its result depends only on its arguments, and it changes nothing outside itself.
- **Three hidden side effects in data code** Mutating arguments, reading globals, reading the clock or randomness inside the function.
- **Return a new DataFrame with an extra column** df.assign(col=...). df["col"] = ... changes the caller's frame.
- **Program shape** Read at the edge, compute in a pure core, write at the edge.

Practice

Exercise 1. Multiple choice (level 1)

Which function is pure?

- a) `def add(rows, x): rows.append(x); return rows`
- b) `def share(n, total): return n / total if total else None`
- c) `def stamp(df): return df.assign(ts=pd.Timestamp.now())`
- d) `def save(df, path): df.to_csv(path)`

Why?

- r1) Its result depends only on its arguments, and it changes nothing outside itself.
- r2) It is the shortest.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3?

```

1 import pandas as pd
2
3
4 def add_flag(df):
5     df["flag"] = df["x"] > 1

```

```

6     return df
7
8
9 original = pd.DataFrame({"x": [1, 2]})
10 result = add_flag(original)
11 print(list(original.columns))

```

Exercise 3. Refactor (level 3)

Make `add_age` pure: take the reference year as a parameter, return a new DataFrame, and leave the input unchanged. Keep the name `add_age` and the new column `age`.

Starter code:

```

1 import datetime
2
3
4 def add_age(df):
5     df["age"] = datetime.date.today().year - df["birth_year"]
6     return df

```

Exercise 4. Spot the bug (level 4)

The second call returns a list containing the first call's flags. Tap the line, then fix the function without changing its behaviour otherwise.

```

1 def flag_low_bases(bases: dict[str, float], minimum: float, flags:
    list[str] = []) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     for name, base in bases.items():
4         if base < minimum:
5             flags.append(name)
6     return flags

```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Sort each step of a weekly scoring script into the pure core or the edge.

Options: Edge, Pure core.

1. Read the respondent extract from storage
2. Compute features from the extract and a reference date
3. Score respondents with a model object passed in
4. Write scores to the output table and log the row count

Sources

- In pandas 3 Copy-on-Write is always on, but assigning a column changes the DataFrame it is called on. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository (predict-output exercise).)

B2-L2 Dataclasses as configuration objects

Concept

Analysis code needs settings: a minimum base size, a weight column, a list of waves. Passing them around as a loose `dict` has three costs. A typo in a key (`"min_bse"`) fails late or not at all. Nothing states which keys exist. Any function can change a value half way through a run.

A **dataclass** (Python 3.7 and later) turns the settings into a named, typed object.

```
1 from dataclasses import dataclass, field
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     weight_col: str
7     min_base: int = 50
8     waves: tuple[str, ...] = ()
9
10     def __post_init__(self) -> None:
11         if self.min_base <= 0:
12             raise ValueError(f"min_base must be positive, got {self.min_base}")
```

What you get:

- **Named fields.** `cfg.min_bse` raises `AttributeError` at once. Your editor and mypy see the fields.
- `frozen=True`. Assigning a field raises `dataclasses.FrozenInstanceError`. Settings stay fixed for the run. Use `dataclasses.replace(cfg, min_base=30)` to build a changed copy.
- `__post_init__`. Runs after the generated `__init__`. Put checks here so a bad config fails at load time, not in hour three.

Two traps:

- **Type hints are not checked at runtime.** `ReportConfig(weight_col="w", min_base="30")` builds without error. Convert or check types in `__post_init__`, or use `pydantic` (lesson B5-L2) when input comes from files.

- **Mutable defaults.** `waves: list[str] = []` raises `ValueError` when the class is defined. Use `field(default_factory=list)`, or an immutable tuple.

Load the config once at the edge, from YAML, JSON or arguments, and pass the object to the pure core.

Why it matters

A weighting script reads `config["min_base"]` from a dict built in a notebook cell. An analyst edits the cell to `config["minbase"] = 30` for a test and forgets to revert. The script keeps using the old default of 50, so 14 subgroups the client asked to see are suppressed in the published tables. Nobody notices for two waves.

With a frozen dataclass, `ReportConfig(minbase=30)` raises `TypeError: unexpected keyword argument` at the first run.

Worked example: weak code

```

1 CONFIG = {"weight_col": "weight", "min_base": 50, "waves": []}
2
3
4 def suppress_small(counts, config=CONFIG):
5     config["min_base"] = config.get("minbase", 50)
6     return {k: v for k, v in counts.items() if v >= config["min_base"]}

```

1. **Line 1.** A module-level dict: no fixed keys, no types, shared and mutable.
2. **Line 4.** A mutable default that every call shares.
3. **Line 5.** A typo in the key silently falls back to 50, and the write changes the shared dict.

Worked example: strong code

```

1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     """Settings for one reporting run."""
7
8     weight_col: str
9     min_base: int = 50
10
11     def __post_init__(self) -> None:
12         if not isinstance(self.min_base, int) or self.min_base <= 0:
13             raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")
14
15

```

```

3 def suppress_small(counts: dict[str, int], config: ReportConfig) ->
    dict[str, int]:
17     """Keep subgroups whose base reaches config.min_base."""
18     return {k: v for k, v in counts.items() if v >= config.min_base}

```

1. **Line 4.** Fixed fields, and no assignment after construction.
2. **Line 12.** Validation at load time. Hints alone are not checked at runtime.
3. **Line 16.** The function receives the config and reads it. It never changes it.

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

Recap card

- **Why a frozen dataclass for settings?** Named, typed fields, unknown keys rejected, and no assignment after construction.
- **Change one setting on a frozen config** `dataclasses.replace(cfg, min_base=30)` returns a new object.
- **Are dataclass type hints checked at runtime?** No. Validate or convert in **post_init**, or use `pydantic` for external input.
- **Empty list default on a dataclass field** `field(default_factory=list)`. A plain `[]` raises `ValueError` at class definition.

Practice

Exercise 1. Multiple choice (level 1)

`cfg` is an instance of a `@dataclass(frozen=True)` class with a field `min_base`. What does `cfg.min_base = 30` do?

- a) Sets `min_base` to 30.
- b) Raises `dataclasses.FrozenInstanceError`.
- c) Creates a copy with `min_base` 30.
- d) Does nothing, silently.

Why?

- r1) `frozen=True` makes the generated **setattr** raise on every field assignment.
- r2) Integer fields are read-only in every dataclass.

Exercise 2. Predict the output (level 2)

What does this print?

```

1 from dataclasses import dataclass
2
3

```

```

4 @dataclass
5 class Settings:
6     min_base: int = 50
7
8
9 print(Settings("30"))

```

Exercise 3. Fill the gap (level 2)

Give `waves` an empty list as its default, safely.

```

1 from dataclasses import dataclass, field
2
3
4 @dataclass
5 class Run:
6     waves: list[str] = field(____(1)=list)

```

Exercise 4. Write the function (level 3)

Complete `ReportConfig` and write `load_config(raw)`.

- `ReportConfig` is a frozen dataclass with `weight_col: str` and `min_base: int = 50`.
- In `__post_init__`, raise `ValueError` if `min_base` is not an `int` or is not positive.
- `load_config(raw: dict[str, object]) -> ReportConfig` builds the config from a dict. An unknown key must raise `TypeError`, a missing `weight_col` must raise `TypeError`.

Starter code:

```

1 from dataclasses import dataclass
2
3
4 class ReportConfig:
5     ...
6
7
8 def load_config(raw):
9     ...

```

Exercise 5. Architecture design (level 4)

A weighting pipeline reads its settings from `weighting.yaml`. Choose how the settings flow through the code.

Options: A frozen dataclass with typed fields, The dict returned by `yaml.safe_load`, Module-level constants set from the file at import.

1. Shape of the settings in code
2. When values are checked

3. How functions get the settings

Sources

- Assigning a field of a frozen dataclass raises `dataclasses.FrozenInstanceError`, a subclass of `AttributeError`. (verified: Python dataclasses documentation; run on Python 3.11 and 3.14 in this repository.)
- A list default on a dataclass field raises `ValueError` at class definition. (verified: Python dataclasses documentation; run on Python 3.11 in this repository.)
- Dataclasses do not check or convert field types at runtime. (verified: Python dataclasses documentation; predict-output exercise run in this repository.)

B2-L3 When a class earns its place

Concept

Python does not require classes. Many data science classes are functions in disguise: an `__init__` that stores arguments, and one method that uses them.

```
1 class Normaliser:
2     def __init__(self, col):
3         self.col = col
4
5     def run(self, df):
6         return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

Every caller writes `Normaliser("spend").run(df)`. A function, `normalise(df, col)`, says the same thing in one call, with nothing to construct.

A class earns its place when at least one of these holds:

- **State that outlives one call.** A weighting model learns targets in `fit` and applies them in `transform`. scikit-learn's `StandardScaler` stores `mean_` and `scale_` between the two calls.
- **An invariant to protect.** A `Weights` object guarantees values are positive and sum to the sample size. Code that builds one through its constructor cannot break that rule.
- **Several operations on the same data.** A `SurveyDesign` with `mean()`, `ci()` and `subgroup()` that all need weights, strata and clusters.

- **A shared interface.** Several classes with the same method, such as `load()`, so the caller swaps one for another.

Settings with no behaviour belong in a dataclass (lesson B2-L2). A single step belongs in a function (lesson B2-L1).

Test a design by asking: if you delete the class and keep its methods as functions, what breaks? If nothing, keep the functions.

Why it matters

A team wraps each cleaning step in a class: `DropDuplicates`, `FixDates`, `RenameColumns`, each with `__init__` and `run`. A new analyst adds a `self.seen_ids` set to `DropDuplicates` to speed up a second call. The pipeline object is reused across two tracker waves, so wave 2 silently drops every respondent who also took part in wave 1: 38% of the sample.

Pure functions have no `self` to keep state state between calls.

Worked example: weak code

```

1 class AgeBander:
2     def __init__(self, bins, labels):
3         self.bins = bins
4         self.labels = labels
5
6     def run(self, df):
7         return df.assign(age_band=pd.cut(df["age"], self.bins,
8             labels=self.labels))
9
10 banded = AgeBander([0, 34, 54, 120], ["18-34", "35-54", "55+"]).run(df)

```

1. **Line 1.** No state outlives a call, no invariant, one operation: a function in disguise.
2. **Line 6.** A single method named `run` is the usual sign.
3. **Line 10.** Every caller builds an object only to call one method.

Worked example: strong code

```

1 import pandas as pd
2
3
4 ① def add_age_band(df: pd.DataFrame, bins: list[int], labels: list[str]) ->
    pd.DataFrame:
5     """Return df with age_band cut from age."""
6     return df.assign(age_band=pd.cut(df["age"], bins, labels=labels))
7
8
9 class RakingWeights:

```

```

10     """Learn raking targets on one wave, apply them to later waves."""
11
12     def __init__(self, targets: dict[str, dict[str, float]]) -> None:
13         self.targets = targets
14         self.factors_: dict[str, float] | None = None
15
16     def fit(self, df: pd.DataFrame) -> "RakingWeights":
17         ...
18         return self
19
20     def transform(self, df: pd.DataFrame) -> pd.Series:
21         if self.factors_ is None:
22             raise RuntimeError("call fit before transform")
23         ...

```

1. **Line 4.** One step, no state: a function.
2. **Line 14.** State that outlives one call, learned in fit and used in transform.
3. **Line 22.** The class protects an order of operations a function could not.

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

Recap card

- **A class with init and one run method** A function in disguise. Replace it with a function.
- **Four reasons a class earns its place** State across calls, an invariant to protect, several operations on the same data, a shared interface.
- **The deletion test** Turn the methods into functions. If nothing breaks, keep the functions.
- **Settings with no behaviour** A frozen dataclass.

Practice

Exercise 1. Multiple choice (level 1)

Which of these most needs a class?

- a) Rename columns using a mapping.
- b) Learn category frequencies on training data and encode test data with them.
- c) Convert a date column to UTC.
- d) Hold three settings with no behaviour.

Why?

- r1) State learned in one call must be reused in a later call.
- r2) It has the most lines of code.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 class Deduper:
2     def __init__(self) -> None:
3         self.seen: set[int] = set()
4
5     def run(self, ids: list[int]) -> list[int]:
6         out = [i for i in ids if i not in self.seen]
7         self.seen.update(ids)
8         return out
9
10
11 d = Deduper()
12 d.run([1, 2, 3])
13 print(d.run([2, 3, 4]))
```

Exercise 3. Refactor (level 3)

Replace the class with a function `add_share(df, col)` that returns a new DataFrame with a column `share` equal to `df[col] / df[col].sum()`. If the column sums to zero, set `share` to `0.0` for every row.

Starter code:

```
1 class ShareAdder:
2     def __init__(self, col):
3         self.col = col
4
5     def run(self, df):
6         return df.assign(share=df[self.col] / df[self.col].sum())
```

Exercise 4. Spot the bug (level 4)

The same `Scaler` object is fitted on wave 1 and used for waves 2 and 3. Wave 3 results match a scaler fitted on wave 2 only. Tap the line that causes it.

```
1 class Scaler:
2     """Standardise with the mean and sd learned in fit."""
3
4     def fit(self, values: list[float]) -> "Scaler":
5         n = len(values)
6         self.mean_ = sum(values) / n
7         self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8         return self
9
10    def transform(self, values: list[float]) -> list[float]:
11        self.fit(values)
12        return [(v - self.mean_) / self.sd_ for v in values]
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Architecture design (level 5)

Choose a form for each piece of a survey analysis package.

Options: Function, Frozen dataclass, Class with methods.

1. Recode a Likert column to numbers using a mapping
2. Confidence level, minimum base and weight column
3. Weights, strata and clusters used by mean, CI and subgroup methods

Sources

- scikit-learn's StandardScaler stores mean_ and scale_ after fit. (verified: scikit-learn 1.8 API reference; checked in this repository's dev environment.)

B3 Clean pandas

Roadmap: Prerequisite for all months. Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L1 Dtypes in pandas 3, nullable and string

Concept

Every pandas column has a **dtype**. The wrong dtype gives wrong answers without an error: IDs that lose leading zeros, counts that turn into floats, text that sorts as numbers.

Three facts about pandas 3.0:

- **Text columns default to the `str` dtype.** `pd.Series(["a", None]).dtype` prints `str`, not `object`. Missing values in it are `NaN`. Only strings are allowed, so a stray integer no longer hides in a text column.
- **A missing value turns an integer column into `float64`,** unless you ask for a nullable dtype. `pd.Series([1, None])` is `float64`. `pd.Series([1, None], dtype="Int64")` keeps integers and shows the gap as `<NA>`.
- **Setting a value that does not fit raises.** Assigning `1.5` into an `int64` Series raises `TypeError` instead of silently upcasting the column. (Assigning `NaN` still upcasts to `float64`.)

The capital letter matters. `int64` is NumPy's integer with no missing values. `Int64`, `Float64` and `boolean` are pandas **nullable** dtypes that hold `pd.NA`.

Set dtypes where data enters, not later:

```
1 df = pd.read_csv(  
2     path,  
3     dtype={"respondent_id": "str", "postcode": "str", "household_size":  
4         "Int64"},  
5 )
```

Without the `dtype` map, `read_csv` reads a postcode such as `01234` as the number `1234.0` once any row is blank. With it, `"01234"` stays a string.

Check the result with `df.dtypes` right after loading. `df.convert_dtypes()` switches every column to the nullable equivalents in one call, useful for exploration. An explicit map is better in pipelines, because it fails loudly when the source changes.

Why it matters

A panel file stores respondent IDs such as 00731. One wave has a blank ID row, so `read_csv` infers `float64` and the ID becomes 731.0. The join to the previous wave, where IDs were read as text, matches nothing. The longitudinal report shows 0% retention for a panel that kept 82% of its members.

`dtype={"respondent_id": "str"}` at load time keeps 00731 as written in every wave.

Worked example: weak code

```
❶ df = pd.read_csv("panel_w2.csv")
❷ df["respondent_id"] = df["respondent_id"].astype(int).astype(str)
❸ df["household_size"] = df["household_size"].fillna(0).astype(int)
```

1. **Line 1.** Dtypes inferred: a blank row turns IDs and counts into float64.
2. **Line 2.** Leading zeros were lost at load time. This cannot bring them back.
3. **Line 3.** Replaces 'unknown' with 0, a real value that biases every mean.

Worked example: strong code

```
1 import pandas as pd
2
3 PANEL_DTYPES: dict[str, str] = {
❶     "respondent_id": "str",
❷     "household_size": "Int64",
6 }
7
8
9 def read_panel(path: str) -> pd.DataFrame:
10     """Read one panel wave with explicit dtypes."""
❸     return pd.read_csv(path, dtype=PANEL_DTYPES)
```

1. **Line 4.** IDs are labels, not numbers: 00731 stays 00731.
2. **Line 5.** Nullable integer: a gap stays <NA>, the rest stay integers.
3. **Line 11.** Dtypes fixed where data enters. In a pipeline, load the map from config.

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

Recap card

- **Default text dtype in pandas 3.0** str, with NaN for missing values. Only strings are allowed.
- **int64 vs Int64** int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
- **Keep leading zeros in IDs** Read the column with dtype str in `read_csv`.

- **Assign 1.5 into an int64 Series on pandas 3** Raises `TypeError` instead of silently upcasting.
- **Where to set dtypes** At load time, from a dtype map in config. Check `df.dtypes` right after.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, what is the dtype of `pd.Series([4, None, 2])`?

- a) `int64`
- b) `Int64`
- c) `float64`
- d) `object`

Why?

- r1) NumPy `int64` cannot hold a missing value, so pandas stores `NaN` in a float column.
- r2) pandas 3 stores every number as float.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```
1 import io
2
3 import pandas as pd
4
5 csv = "id,postcode\n1,01234\n2,\n"
6 df = pd.read_csv(io.StringIO(csv))
7 print(df["postcode"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Keep IDs as text and let the count column hold missing values as integers.

```
1 df = pd.read_csv(path, dtype={"respondent_id": "____(1)",
    "household_size": "____(2)"})
```

Exercise 4. Write the function (level 3)

Write `read_panel(text: str) -> pd.DataFrame`. It reads CSV text with columns `respondent_id` and `household_size`. Keep `respondent_id` as text with leading zeros, and `household_size` as a nullable integer where blanks become `<NA>`. Take the dtype map from a module constant `PANEL_DTYPES`.

Starter code:

```

1 import io
2
3 import pandas as pd
4
5
6 def read_panel(text):
7     return pd.read_csv(io.StringIO(text))

```

Exercise 5. Spot the bug (level 4)

The mean household size reported for wave 3 dropped from 2.6 to 2.1. Nothing changed in the sample. Tap the line that caused it.

```

1 import pandas as pd
2
3
4 def mean_household(df: pd.DataFrame) -> float:
5     """Mean household size among respondents who answered."""
6     sizes = df["household_size"].astype("Int64")
7     sizes = sizes.fillna(0)
8     return float(sizes.mean())

```

Write down the line number of the bug. Then write the fix.

Sources

- In pandas 3.0 text columns default to the str dtype, with NaN as the missing value. (verified: pandas 3.0 release notes; run on pandas 3.0.6 and 3.0.2 in this repository.)
- In pandas 3.0 setting an incompatible value such as 1.5 into an int64 Series raises TypeError. (verified: pandas 3.0 release notes (setitem upcasting removed); run on pandas 3.0.6 and 3.0.2 in this repository.)
- read_csv infers float64 for an integer-like column with a blank value. (verified: Run on pandas 3.0.6 and 3.0.2 in this repository.)

B3-L2 Vectorise instead of apply

Concept

pandas stores each column as an array. A **vectorised** operation, such as `df["a"] * df["b"]`, `np.where()` or `pd.cut()`, runs one compiled loop over the whole array. `df.apply(func, axis=1)` does the opposite. It builds a Series for each row and calls your Python function once per row.

The gap is large. On one million rows, a row-wise `apply` that picks a weight when a flag is set took 4.0 seconds. `np.where` did the same job in 7.5

milliseconds, about 500 times faster. (pandas 3.0.6, NumPy 2.4.6, one laptop core.)

Speed is not the only gain. Vectorised code states the rule for the whole column in one line, so a reviewer checks one expression instead of a branching function.

Reach for these first:

Task	Vectorised tool
Arithmetic between columns	<code>+, -, *, /</code> on Series
Two-way choice	<code>np.where(cond, a, b)</code>
Many-way choice	<code>np.select(conditions, choices, default)</code>
Numeric bands	<code>pd.cut(values, bins, labels, right=False)</code>
Lookup from a dict	<code>Series.map(mapping)</code>
Limits	<code>Series.clip(lower, upper)</code>

Watch missing values. `NaN > 60` is `False`, so `np.where(minutes > 60, "heavy", "light")` labels a missing value "light". Handle missing values on purpose: `pd.cut` keeps `NaN` as missing, and `Series.where(values.notna())` puts it back after `np.where`.

Keep `apply` for work with no vectorised form, such as calling an external parser on each string. Even then, try `Series.map` or a list comprehension on one column before `apply(axis=1)`.

Why it matters

A weighting script derives 25 columns with `apply(axis=1)`. It ran in 8 minutes on 2 million rows, so nobody noticed. The programme then merged three trackers into one file of 40 million rows. At the measured rate of about 4 seconds per million rows per column, the derivations alone need roughly 67 minutes. The job hit its 60-minute timeout every night for a week, and weighted results for all three trackers were late.

The same script also labelled respondents with a missing `minutes` value as "light" viewers, because the row function used `if row["minutes"] > 60`. The vectorised rewrite fixed the speed, and review of the one-line rule exposed the missing-value bug.

Worked example: weak code

```
1 import pandas as pd
2
```

```

3
1 def derive(df):
4     def per_row(row):
5         weighted_reach = row["final_weight"] if row["reach_flag"] == 1
6         else 0.0
2     if row["age"] < 35:
7         band = "16-34"
8     elif row["age"] < 55:
9         band = "35-54"
10    else:
11        band = "55+"
12    return pd.Series({"weighted_reach": weighted_reach, "age_group":
13        band})
14
4 df[["weighted_reach", "age_group"]] = df.apply(per_row, axis=1)
16 return df

```

- Line 4.** No type hints and no docstring. The reader cannot tell what the frame must contain.
- Line 7.** `NaN < 35` is `False`, so a missing age falls through to `55+` without a warning.
- Line 15.** Builds a Series and calls Python for every row. About 500 times slower than a vectorised rule.
- Line 15.** Mutates the caller's frame. A pure function returns a new frame.

Worked example: strong code

```

1 import numpy as np
2 import pandas as pd
3
1 AGE_BINS = [16, 35, 55, np.inf]
4 AGE_LABELS = ["16-34", "35-54", "55+"]
5
6
7
8 def derive(df: pd.DataFrame) -> pd.DataFrame:
9     """Add weighted reach and age group.
10
11     Args:
12         df: Respondents with reach_flag, final_weight and age.
13
14     Returns:
15         A new frame. Missing ages give a missing age_group.
16     """
2     return df.assign(
3         weighted_reach=np.where(df["reach_flag"] == 1,
17         df["final_weight"], 0.0),
4         age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
18         right=False),

```

1. **Line 4.** Band edges live in one constant, so a change to the bands is a one-line change.
2. **Line 17.** `assign` returns a new frame, so the caller's data is untouched.
3. **Line 18.** One compiled pass over the column.
4. **Line 19.** `right=False` makes each band include its lower edge: 35 goes to 35-54. Missing ages stay missing.

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Recap card

- **Why is `df.apply(func, axis=1)` slow?** It calls a Python function once per row. `np.where` measured about 500 times faster on one million rows.
- **Vectorised if-else in NumPy** `np.where(cond, value_if_true, value_if_false)`. For several conditions, `np.select`.
- **What does `NaN > 60` return?** False. `np.where` and `np.select` send missing values to a real category unless you handle them.
- **Numeric bands that keep missing values missing** `pd.cut(values, bins, labels, right=False)`.

Practice

Exercise 1. Multiple choice (level 1)

Which line computes `weighted_reach` for a frame with one million rows in a single compiled pass?

- a) `df.apply(lambda r: r["final_weight"] if r["reach_flag"] == 1 else 0.0, axis=1)`
- b) `[w if f == 1 else 0.0 for w, f in zip(df["final_weight"], df["reach_flag"])]`
- c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`
- d) `for i, row in df.iterrows(): ...`

Why?

- r1) It evaluates the condition and the choice on whole arrays in compiled code.
- r2) It uses fewer characters than the other options.
- r3) NumPy runs the lambda on several cores.

Exercise 2. Predict the output (level 2)

What does this print?

```
1 import pandas as pd
2
3 s = pd.Series([0.0, 1.5, None])
4 print((s > 1).tolist())
```

Exercise 3. Fill the gap (level 2)

Complete the line so rows with `reach_flag == 1` keep their weight and every other row gets zero.

```
1 df["weighted_reach"] = np.__(1)(df["reach_flag"] == 1,
    df["final_weight"], ____(2))
```

Exercise 4. Write the function (level 3)

Write `weighted_reach_pct(df)`. It returns the weighted share of rows with `reach_flag == 1`: the sum of `final_weight` where the flag is 1, divided by the sum of all `final_weight`. Return `None` when the total weight is zero or the frame is empty. Do not use `apply` or a loop over rows.

Starter code:

```
1 import numpy as np
2 import pandas as pd
3
4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     ...
```

Exercise 5. Refactor (level 4)

Refactor `add_age_group` to vectorised code. Keep the bands: under 35 is 16-34, under 55 is 35-54, anything else is 55+. A missing age must give a missing age group, not 55+. Do not change the input frame.

Starter code:

```
1 import pandas as pd
2
3
4 def add_age_group(df):
5     def band(row):
6         if row["age"] < 35:
7             return "16-34"
8         if row["age"] < 55:
9             return "35-54"
10        return "55+"
11
12    df["age_group"] = df.apply(band, axis=1)
```

```
13     return df
```

Exercise 6. Spot the bug (level 5)

The docstring promises that missing minutes stay missing. A respondent with no minutes recorded shows as "light". Tap the line with the bug, then fix the function.

```
1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
   minutes stay missing."""
3     minutes = df["minutes"]
4     band = np.where(minutes >= 60, "heavy", "light")
5     return df.assign(viewing_band=band)
```

Write down the line number of the bug. Then write the fix.

Sources

- Row-wise apply was about 500 times slower than np.where on one million rows. (verified: Measured in this repo's build environment: pandas 3.0.6, NumPy 2.4.6, 4.03 s against 7.5 ms.)
- pd.cut keeps NaN as missing and supports right=False. (verified: Run in pandas 3.0.6.)

B3-L3 Method chaining and Copy-on-Write

Concept

Copy-on-Write (CoW) is always on in pandas 3.0. Every indexing result, such as `df[cols]`, `df[mask]` or `df["a"]`, behaves as a separate object. Changing it never changes `df`. pandas copies data only when you write to a shared object, so reads stay cheap.

One consequence: **chained assignment never works**.

```
1 df[df["age"] < 18]["weight"] = 0      # df unchanged,
   ChainedAssignmentError warning
2 df.loc[df["age"] < 18, "weight"] = 0  # one step: df changed
```

The first line writes to a temporary copy, which Python then discards. pandas 3 emits a `ChainedAssignmentError` warning, but the code keeps running. `SettingWithCopyWarning` no longer exists.

CoW rewards a second habit: **method chaining**. Build the result as one expression of steps that each return a new frame.

```
1 result = (
2     raw
```

```

3     .rename(columns=COLUMN_MAP)
4     .loc[lambda d: d["age"] >= 18]
5     .assign(weight=lambda d: d["weight"].clip(upper=5.0))
6     .pipe(add_age_band, bins=BINS, labels=LABELS)
7 )

```

- `assign(col=lambda d: ...)` refers to the frame at that step, not the original `raw`.
- `.loc[lambda d: ...]` filters on the current step.
- `.pipe(func, **kwargs)` calls your own pure function (lesson B2-L1) inside the chain.

Chains read top to bottom, leave no half-finished variables (`df2`, `df_tmp`, `df_final`) in the namespace, and do not mutate the input. Keep each chain to about ten steps. Split longer ones into named functions and join them with `pipe`.

To debug a chain, comment out steps from the bottom up, or add `.pipe(lambda d: print(d.shape) or d)` between steps.

Why it matters

An analyst caps extreme weights with `df[df["weight"] > 5]["weight"] = 5`. On pandas 2 it printed a `SettingWithCopyWarning` that everyone had learned to ignore. The weights were never capped. Three respondents with weights above 12 moved the reported brand share by 4 points. After the upgrade to pandas 3 the assignment still does nothing, with a different warning.

`df.assign(weight=df["weight"].clip(upper=5.0))` or a single `df.loc[mask, "weight"] = 5` does what the analyst meant.

Worked example: weak code

```

1 df = raw.rename(columns=COLUMN_MAP)
2 df2 = df[df["age"] >= 18]
3 df2[df2["weight"] > 5]["weight"] = 5.0
4 df3 = add_age_band(df2, BINS, LABELS)
5 result = df3

```

1. **Line 2.** Intermediate names that outlive their use and invite edits to the wrong one.
2. **Line 3.** Chained assignment: writes to a temporary copy. The weights are not capped.
3. **Line 5.** Which of `df`, `df2`, `df3` is the final state? The reader has to trace every line.

Worked example: strong code

```
1 import pandas as pd
2
3
4 def prepare(raw: pd.DataFrame, column_map: dict[str, str], bins:
    list[int], labels: list[str]) -> pd.DataFrame:
5     """Rename, keep adults, cap weights at 5 and add age bands."""
6     return (
7         raw
8         .rename(columns=column_map)
9         .loc[lambda d: d["age"] >= 18]
10        .assign(weight=lambda d: d["weight"].clip(upper=5.0))
11        .pipe(add_age_band, bins=bins, labels=labels)
12    )
```

1. **Line 9.** Filters the renamed frame at this step.
2. **Line 10.** Returns a new frame with capped weights. No assignment to a copy.
3. **Line 11.** Your own pure function, called as one step of the chain.

Under Copy-on-Write, write with `.loc` in one step, or better, build a new frame with a chain of `assign`, `loc` and `pipe`.

Recap card

- **Copy-on-Write in pandas 3.0** Always on. Indexing results behave as separate objects, copied only on write.
- **`df[mask]["col"] = value` on pandas 3** Changes nothing and emits `ChainedAssignmentError`. Use `df.loc[mask, "col"] = value`.
- **Refer to the current step inside a chain** A `lambda`: `assign(x=lambda d: ...)` or `loc[lambda d: ...]`.
- **Call your own function inside a chain** `.pipe(func, **kwargs)`.

Practice

Exercise 1. Multiple choice (level 1)

On pandas 3.0, `df[df["age"] < 18]["weight"] = 0` runs. What happens to `df`?

- a) Weights of under-18s become 0.
- b) Nothing. pandas emits a `ChainedAssignmentError` warning.
- c) It raises an exception and stops.
- d) Every weight becomes 0.

Why?

- r1) The first indexing step returns a new object, so the write lands on a temporary copy.

- r2) pandas 3 makes every DataFrame read-only.

Exercise 2. Predict the output (level 2)

What does this print on pandas 3.0?

```
1 import pandas as pd
2
3 df = pd.DataFrame({"a": [1, 2, 3]})
4 s = df["a"]
5 s.iloc[0] = 100
6 print(df["a"].tolist())
```

Exercise 3. Put in order (level 2)

Put the steps of the chain in order. The raw columns are `wt` and `age`. Weights must average 1.0 across the adults in the output.

- `.pipe(add_age_band, bins=BINS, labels=LABELS)`
- `.assign(weight=lambda d: d["weight"] / d["weight"].mean())`
- `.rename(columns={"wt": "weight"})`
- `.loc[lambda d: d["age"] >= 18]`

Exercise 4. Refactor (level 3)

Rewrite `cap_weights` as one method chain that returns a new frame. Keep only rows where `age >= 18`, then cap `weight` at `cap`. Leave the input unchanged. No chained assignment.

Starter code:

```
1 def cap_weights(df, cap):
2     adults = df[df["age"] >= 18]
3     adults[adults["weight"] > cap]["weight"] = cap
4     return adults
```

Exercise 5. Spot the bug (level 4)

Weights for invalid interviews should be zero. The check after the function shows they are unchanged. Tap the line to fix, and rewrite the function.

```
1 import pandas as pd
2
3
4 def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
5     """Set weight to 0 where valid is False."""
6     out = df.copy()
7     out[~out["valid"]]["weight"] = 0.0
8     return out
```

Write down the line number of the bug. Then write the fix.

Sources

- Copy-on-Write is always enabled in pandas 3.0 and the `mode.copy_on_write` option has no effect. (verified: pandas 3.0 release notes; Pandas4Warning text observed on pandas 3.0.6 and 3.0.2 in this repository.)
- Chained assignment never updates the original in pandas 3.0 and emits `ChainedAssignmentError` as a warning. (verified: pandas Copy-on-Write user guide; run on pandas 3.0.6 and 3.0.2 in this repository.)
- `SettingWithCopyWarning` no longer exists in pandas 3.0. (verified: pandas 3.0 release notes; `pandas.errors` has no `SettingWithCopyWarning` on 3.0.6.)

Track B best-practice checklist

- ☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.
- ☐ ruff and mypy pass before every commit.
- ☐ Functions are pure where possible. Side effects sit at the edges.
- ☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.
- ☐ Inputs are validated with pandera or pydantic at every boundary.
- ☐ Seeds, config and data versions are recorded for every result.
- ☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.
- ☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.
- ☐ The metric matches the decision. Calibration and subgroup performance are checked.
- ☐ Every experiment is tracked in MLflow.
- ☐ Tests cover edge cases, including empty inputs and zeros.
- ☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B1-L1 One pyproject.toml and a src layout

B1-L1 exercise 1

Hints. (1) Which table never ships in the built package? (2) It was added by PEP 735. (3) `[dependency-groups]`.

Answer. b) `[dependency-groups]` Because: r1) Dependency groups (PEP 735) are named lists that are not part of the built distribution.

`[dependency-groups]` holds development-only requirements. `pip install --group dev (pip 25.1+)` installs them.

Common wrong answers.

- a: `[project] dependencies` install for every user of the package, in production too.
- c: That table configures how pytest runs. It does not install anything.
- d: A comment installs nothing and drifts from reality.
- b|r2: Right table, wrong reason. Groups can hold any development tools: tests, linting, docs, notebooks.

B1-L1 exercise 2

Hints. (1) The runtime key is plural. (2) The development table has a hyphenated name. (3) `dependencies` and `dependency-groups`.

Answer.

```
1 [project]
2 name = "churn"
3 requires-python = ">=3.11"
4 dependencies = ["pandas>=3.0,<3.1", "scikit-learn>=1.8,<1.10"]
5
6 [dependency-groups]
7 dev = ["pytest>=9", "ruff>=0.16", "mypy>=2.3"]
```

`[project].dependencies` lists what users need to run the package.

`[dependency-groups]` lists what developers need to work on it.

Common wrong answers.

- `g1 = requires`: The runtime list under `[project]` is called `dependencies`.

- `g2 = tool.dev: [tool.*]` tables configure tools. Development requirements go in `[dependency-groups]`.
- `g2 = optional-dependencies:` Optional dependencies (extras) ship with the package for users to opt into. Development-only tools belong in `[dependency-groups]`.

B1-L1 exercise 3

Hints. (1) `tomllib.loads(text)` returns a dict. (2) Use `.get` with defaults for the project table and the dependencies list. (3) Keep entries that contain none of `= < > ~ !`.

Answer.

```
1 import tomllib
2
3 CONSTRAINT_CHARS = set("=<>~!")
4
5
6 def unpinned(pyproject_text: str) -> list[str]:
7     """Runtime dependencies with no version constraint.
8
9     Args:
10         pyproject_text: Contents of pyproject.toml.
11
12     Returns:
13         Sorted requirement strings without =, <, >, ~ or !.
14     """
15     project = tomllib.loads(pyproject_text).get("project", {})
16     deps = project.get("dependencies", [])
17     return sorted(d for d in deps if not CONSTRAINT_CHARS & set(d))
```

`tomllib` ships with Python 3.11+. Checking characters is enough to spot a missing constraint, and `.get` handles projects that declare no dependencies.

B1-L1 exercise 4

Hints. (1) Why can Python not see `src/churn`? (2) What makes a package importable everywhere in the environment? (3) An editable install.

Answer. b) Run `pip install -e .` in the project's virtual environment.

Because: r1) The `src` layout is designed so that only the installed package is importable, which makes tests use it like any user would.

`pip install -e .` installs the package in editable mode, so imports resolve to `src/churn` and edits take effect without reinstalling.

Common wrong answers.

- a: Path hacks in every file defeat the point of the layout and break when files move.

- `c`: That removes the protection the layout gives: tests would import the working directory instead of the installed package.
- `d`: Copies drift. Tests must exercise the real package.
- `b|r2`: Right fix, wrong reason. `pytest` can import from folders. The `src` layout deliberately hides the package until it is installed.

B1-L1 exercise 5

Hints. (1) What must more than one program import? (2) What should CI run on every change? (3) What must never be cloned?

Answer.

- Shared logic in the package, imported by notebook and jobs.
- One-off exploration in notebooks, which import the package.
- Known-answer checks in tests/.
- Real data in governed storage, never in Git. Keep only small anonymised samples in the repository.

Package for shared logic, notebooks for exploration, tests for checks, governed storage for data.

Common wrong answers.

- `features = notebook`: The scoring job cannot import a notebook cell, so it would copy it, and the copies would drift.
- `extract = src`: Client data in the repository leaks with every clone. Keep it in governed storage.
- `known_answer = notebook`: A check in a notebook runs only when someone reruns the notebook. Put it in tests/ so CI runs it.

B1-L2 One virtual environment per project, and `python -m pip`

B1-L2 exercise 1

Hints. (1) How many Pythons might be on your PATH? (2) Which `pip` does `python -m pip` use? (3) The one belonging to that python.

Answer. b) It installs into the interpreter that `python` runs, whatever is first on PATH. Because: r1) `python -m pip` runs `pip` as a module of that exact interpreter.

`python -m pip` ties the install to the interpreter you will run. Bare `pip` is whichever `pip` script PATH finds first.

Common wrong answers.

- `a`: Speed is the same. The benefit is installing into the right interpreter.
- `c`: It installs into the current interpreter's environment, which inside a venv is the project only.
- `d`: Resolution is identical. Only the target interpreter is guaranteed.
- `b|r2`: Right answer, wrong reason. Bare pip still works. It may simply belong to a different Python.

B1-L2 exercise 2

Hints. (1) What does `sys.prefix` point to inside a venv? (2) And `sys.base_prefix`? (3) They are different folders.

Answer.

```
1 True
```

`sys.prefix` is the virtual environment, `sys.base_prefix` is the Python it was created from. They differ only inside a venv.

Common wrong answers.

- `False`: Inside a venv, `sys.prefix` points at the venv folder and `sys.base_prefix` at the base installation, so they differ.

B1-L2 exercise 3

Hints. (1) The standard library module that creates environments. (2) The flag for an editable install. (3) `venv` and `-e`.

Answer.

```
1 python -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -e . --group dev
```

`python -m venv .venv` creates the environment. `pip install -e .` installs the project so edits under `src/` take effect immediately.

Common wrong answers.

- `g1 = virtualenv`: `virtualenv` is a separate package. The standard library module is `venv`.
- `g2 = -r: -r` reads a requirements file. Editable installs of the project use `-e .`.

B1-L2 exercise 4

Hints. (1) Inside a venv the two prefixes differ. (2) Raise when they are equal. (3) Read them inside the function body.

Answer.

```
1 import sys
2
3
4 def require_virtualenv() -> None:
5     """Stop when not running inside a virtual environment.
6
7     Raises:
8         RuntimeError: If sys.prefix equals sys.base_prefix.
9     """
10    if sys.prefix == sys.base_prefix:
11        raise RuntimeError("activate the project's virtual environment
    first")
```

Equal prefixes mean the system or base interpreter is running. Reading them at call time keeps the check correct in tests and in any process that changes them.

B1-L2 exercise 5

Hints. (1) Can one environment hold both constraints? (2) Where will the next developer look? (3) CI must match how each environment is used.

Answer.

- Two environments, because the constraint is real and tested.
- Two dependency groups in pyproject.toml, with the reason written next to the pin.
- Two CI jobs, so each environment is tested exactly as it is used.

When two tools conflict, give each its own pinned environment, record why in pyproject.toml, and test both in CI.

Common wrong answers.

- `envs = one_latest`: PySpark 4.0.4 fails with pandas 3.x: `assertDataFrameEqual` raises `ImportError`. One environment cannot satisfy both.
- `envs = one_old`: That works, but the analysis code then runs on an older pandas than intended. Separate environments keep each part current.
- `ci = one_job`: Installing both groups together recreates the conflict in CI.
- `declare = readme`: Chat messages are lost. Record the pin and its reason in pyproject.toml.

B1-L3 ruff and mypy in daily work

B1-L3 exercise 1

Hints. (1) How long is each list? (2) What does zip do when lengths differ? (3) It stops at the shortest.

Answer.

```
1 [(1, 2.0), (0, 1.0)]
```

zip stops at the shortest input, so the third respondent disappears. ruff's B905 asks for `strict=True`, which raises `ValueError` on unequal lengths (Python 3.10+).

Common wrong answers.

- `ValueError`: Only `zip(..., strict=True)` raises. Plain zip stops at the shorter input without a word.
- `[(1, 2.0), (0, 1.0), (1, None)]`: That is `itertools.zip_longest`. zip drops the extra element.

B1-L3 exercise 2

Hints. (1) When can share be None? (2) Is that case possible in real data? (3) Handle it.

Answer. b) Handle the None case before the multiplication. Because: r1) mypy found a real path where the value is None. Handling it removes the crash.

mypy found a reachable crash. Handle `None` explicitly, for example by returning `"n/a"`.

Common wrong answers.

- a: The ignore hides the warning and keeps the crash.
- c: The hint would then lie, and mypy would flag the `return None` instead. The crash stays.
- d: Weakening the checker hides this bug and every future one like it.
- b|r2: Right fix, wrong reason. This is a real crash on zero weight, not style.

B1-L3 exercise 3

Hints. (1) The flag that applies safe fixes. (2) The formatter subcommand. (3) The type checker's command.

Answer.

```

1 ruff check --fix .
2 ruff format .
3 mypy

```

`ruff check --fix .` applies safe fixes, `ruff format .` formats, and `mypy` type-checks with the settings in `pyproject.toml`.

Common wrong answers.

- `g1 = --unsafe-fixes`: Unsafe fixes can change behaviour. Use `--fix` for the safe ones, and review the rest by hand.
- `g3 = python -m pytest`: Tests matter, but the step after formatting in this routine is the type checker.

B1-L3 exercise 4

Hints. (1) Annotate: `flags: list[int], weights: list[float]`. (2) `zip(flags, weights, strict=True)`. (3) In report, store the share and return `'n/a'` when it is `None`.

Answer.

```

1 def weighted_share(flags: list[int], weights: list[float]) -> float |
    None:
2     """Weighted share of flagged rows, or None when the total weight is
    0."""
3     total = sum(weights)
4     if total == 0:
5         return None
6     return sum(w for f, w in zip(flags, weights, strict=True) if f ==
    1) / total
7
8
9 def report(flags: list[int], weights: list[float]) -> str:
10     """Share as a percentage string, or 'n/a' when undefined."""
11     share = weighted_share(flags, weights)
12     if share is None:
13         return "n/a"
14     return f"{share * 100:.1f}%"

```

The types make the `None` case visible, `strict=True` stops silent truncation, and `report` handles `None` on purpose.

B1-L3 exercise 5

Hints. (1) Prefer real type information. (2) When there is none, narrow the exception. (3) Never global.

Answer.

- Stubs exist: install them in the dev group, and `mypy` checks your use of the library.

- No stubs: silence only that module with a targeted override.
- Never switch it off globally, or you lose the check for libraries that do ship types.

Install stubs where they exist, override per module where they do not, and keep the global check on.

Common wrong answers.

- `stubs_exist = override`: Stubs exist, so installing them gives you real checks for free.
- `many = global`: A global setting hides missing stubs you could have installed, for every library, forever.

B2-L1 Pure functions, side effects at the edges

B2-L1 exercise 1

Hints. (1) Does the result depend on anything except the arguments? (2) Does the function change anything outside itself? (3) Only one answers no to both.

Answer. b) `def share(n, total): return n / total if total else None`
 Because: r1) Its result depends only on its arguments, and it changes nothing outside itself.

`share` depends only on `n` and `total` and changes nothing. The others mutate, read the clock, or write a file.

Common wrong answers.

- `a`: `append` mutates the caller's list, a side effect.
- `c`: It reads the clock, so the same input gives a different result each call.
- `d`: Writing a file is a side effect. It belongs at the edge.
- `b` | `r2`: Right function, wrong reason. Purity is about inputs and side effects, not length.

B2-L1 exercise 2

Hints. (1) Is `df` a copy or the caller's object? (2) Does column assignment change the object it is called on? (3) It does.

Answer.

```
1 ['x', 'flag']
```

`df` is the same object as `original`. Assigning a column changes it, so the caller sees `flag` appear. Use `df.assign(flag=...)` to return a new frame.

Common wrong answers.

- ['x']: Copy-on-Write protects against changes through derived objects, but `df["flag"] = ...` changes `df` itself, and `df` is the caller's object.

B2-L1 exercise 3

Hints. (1) Add a parameter for the year. (2) Use `df.assign(...)`, which returns a new frame. (3) `return df.assign(age=as_of_year - df["birth_year"])`.

Answer.

```
1 import pandas as pd
2
3
4 def add_age(df: pd.DataFrame, as_of_year: int) -> pd.DataFrame:
5     """Return a copy of df with age = as_of_year - birth_year."""
6     return df.assign(age=as_of_year - df["birth_year"])
```

The year is an input, so tests fix it. `assign` returns a new frame, so callers keep their data.

B2-L1 exercise 4

Hints. (1) When is the default list created? (2) Is it shared between calls? (3) Build a fresh list inside the function.

Answer. The bug is on line 1.

Fixed version:

```
1 def flag_low_bases(bases: dict[str, float], minimum: float) -> list[str]:
2     """Names of subgroups whose base is below minimum."""
3     return [name for name, base in bases.items() if base < minimum]
```

The default list is created once and shared by every call, a hidden state that makes the function impure. Build the list inside, here with a comprehension.

Common wrong answers.

- Line 5: `append` is where the state leaks, but the shared list is created on line 1.

B2-L1 exercise 5

Hints. (1) Does the step touch anything outside Python's memory? (2) Does its result depend only on its arguments? (3) I/O at the edges, logic in the core.

Answer.

- Reading and writing, logging and choosing the reference date: edge.

- Features and scoring, given all inputs as arguments: pure core.

A thin edge does I/O. A pure core does the thinking and gets the tests.

Common wrong answers.

- `read = core`: Reading storage depends on the world outside the function. Keep it at the edge.
- `write = core`: Writing and logging are side effects. Keep them at the edge.
- `features = edge`: Feature logic given its inputs is deterministic. Make it pure so it can be tested and reused.

B2-L2 Dataclasses as configuration objects

B2-L2 exercise 1

Hints. (1) What does `frozen=True` promise? (2) Is the failure silent or loud? (3) The exception is a subclass of `AttributeError`.

Answer. b) Raises `dataclasses.FrozenInstanceError`. Because: `r1` `frozen=True` makes the generated `setattr` raise on every field assignment. Frozen dataclasses raise `dataclasses.FrozenInstanceError` on assignment. Build a changed copy with `dataclasses.replace`.

Common wrong answers.

- `a`: That is the behaviour without `frozen=True`. Frozen instances reject assignment.
- `c`: Assignment never returns a copy. Use `dataclasses.replace(cfg, min_base=30)` for that.
- `d`: It fails loudly, which is the point: settings cannot drift during a run.
- `b|r2`: Right answer, wrong reason. Without `frozen=True` every field, int or not, accepts assignment.

B2-L2 exercise 2

Hints. (1) Does a dataclass check or convert types? (2) What does the generated `repr` show for a string? (3) It uses `repr()` of each value, so strings keep their quotes.

Answer.

```
1 Settings(min_base='30')
```

The hint `int` is for your editor and mypy. At runtime the dataclass stores `'30'`. Check or convert in `__post_init__`.

Common wrong answers.

- `Settings(min_base=30)`: Dataclasses do not convert values. The string '30' is stored as it is.
- `TypeError`: Type hints are not checked at runtime. The call succeeds and stores a string.

B2-L2 exercise 3

Hints. (1) A plain `= []` raises `ValueError` at class definition. (2) You need a function that builds a fresh list per instance. (3) The keyword is `default_factory`.

Answer.

```
1 from dataclasses import dataclass, field
2
3
4 @dataclass
5 class Run:
6     waves: list[str] = field(default_factory=list)
```

`field(default_factory=list)` calls `list()` for each new instance, so no two instances share one list.

Common wrong answers.

- `g1 = default: field(default=list)` stores the list class itself, not a new list.

B2-L2 exercise 4

Hints. (1) `ReportConfig(**raw)` passes each key as a keyword argument. (2) The generated `__init__` raises `TypeError` on an unknown or missing keyword. (3) Check `isinstance(self.min_base, int)` and `self.min_base > 0` in `__post_init__`.

Answer.

```
1 from dataclasses import dataclass
2
3
4 @dataclass(frozen=True)
5 class ReportConfig:
6     """Settings for one reporting run."""
7
8     weight_col: str
9     min_base: int = 50
10
11     def __post_init__(self) -> None:
12         if not isinstance(self.min_base, int) or self.min_base <= 0:
13             raise ValueError(f"min_base must be a positive int, got {self.min_base!r}")
```

```

14
15
16 def load_config(raw: dict[str, object]) -> ReportConfig:
17     """Build a ReportConfig from a parsed YAML or JSON mapping.
18
19     Args:
20         raw: Keys must match ReportConfig fields.
21
22     Returns:
23         A validated, frozen config.
24
25     Raises:
26         TypeError: On an unknown or missing key.
27         ValueError: On an invalid min_base.
28     """
29     return ReportConfig(**raw) # type: ignore[arg-type]

```

Unpacking the dict into the constructor lets the generated `__init__` reject unknown and missing keys. `__post_init__` catches bad values. `frozen=True` stops later edits.

B2-L2 exercise 5

Hints. (1) Which shape rejects a misspelt key? (2) When would you rather see a bad value: now or in hour three? (3) Globals make functions impure (lesson B2-L1).

Answer.

- A frozen dataclass: named fields, unknown keys rejected, no edits during the run.
- Check once at load time in **post_init**, so a bad file fails before any work starts.
- Pass the object as a parameter, so tests build their own config.

Build a frozen, validated object at the edge and pass it into the pure core.

Common wrong answers.

- `shape = dict`: A dict accepts any key, including typos, and any function can change it.
- `shape = globals`: Reading a file at import is a module-level side effect, and every test shares the same values.
- `when = use`: Checks spread across functions fail late, after hours of work, and drift apart.
- `pass = import`: A shared import is a global. Tests cannot give one function a different config.

B2-L3 When a class earns its place

B2-L3 exercise 1

Hints. (1) Which task has two phases? (2) What must survive between those phases? (3) Learned frequencies must be stored between fit and transform.

Answer. b) Learn category frequencies on training data and encode test data with them. Because: r1) State learned in one call must be reused in a later call.

Fit then transform needs stored state, the classic case for a class, as in scikit-learn's encoders.

Common wrong answers.

- a: One step with no state. A function taking the mapping is enough.
- c: One step with no state. A function is enough.
- d: Settings with no behaviour belong in a dataclass, not a full class.
- b|r2: Right answer, wrong reason. Size does not decide it. State across calls does.

B2-L3 exercise 2

Hints. (1) What is in self.seen after the first call? (2) Which of 2, 3 and 4 is not in it? (3) Only 4.

Answer.

```
1 [4]
```

The object keeps state between calls. Reuse it on a new wave and it drops rows the new wave needs.

Common wrong answers.

- [2, 3, 4]: self.seen still holds 1, 2 and 3 from the first call, so 2 and 3 are dropped.

B2-L3 exercise 3

Hints. (1) Move self.col into a parameter. (2) Compute the total once and test it for zero. (3) `df.assign(share=0.0)` when the total is zero.

Answer.

```
1 import pandas as pd
2
3
4 def add_share(df: pd.DataFrame, col: str) -> pd.DataFrame:
```

```

5     """Return df with share = col / column total, or 0.0 when the total is
      zero."""
6     total = df[col].sum()
7     if total == 0:
8         return df.assign(share=0.0)
9     return df.assign(share=df[col] / total)

```

The class held one value and did one thing. A function with two parameters is shorter, and the zero guard stops a NaN column.

B2-L3 exercise 4

Hints. (1) Which state should transform read? (2) Does transform change that state? (3) Refitting inside transform discards what fit learned.

Answer. The bug is on line 11.

Fixed version:

```

1 class Scaler:
2     """Standardise with the mean and sd learned in fit."""
3
4     def fit(self, values: list[float]) -> "Scaler":
5         n = len(values)
6         self.mean_ = sum(values) / n
7         self.sd_ = (sum((v - self.mean_) ** 2 for v in values) / n) ** 0.5
8         return self
9
10    def transform(self, values: list[float]) -> list[float]:
11        return [(v - self.mean_) / self.sd_ for v in values]

```

Refitting in `transform` throws away the learned state, the only reason the class exists. It also leaks test data into the scaling.

Common wrong answers.

- Line 6: The mean formula is correct. The problem is when it is recomputed.
- Line 12: The scaling formula is correct. It uses values that were just overwritten on line 11.

B2-L3 exercise 5

Hints. (1) Does it hold data, do something, or both? (2) Do several operations share the same data? (3) Delete the class: what breaks?

Answer.

- Recoding is one stateless step: a function.
- Settings with no behaviour: a frozen dataclass.

- A survey design shares the same data across several operations and must keep weights valid: a class.

Match the form to the need: function for a step, dataclass for settings, class for shared state with behaviour.

Common wrong answers.

- `recode = class`: One step, no state that outlives the call. A class adds construction with no benefit.
- `settings = class`: No behaviour to attach. A frozen dataclass gives fields, equality and immutability for free.
- `design = function`: Every function would need weights, strata and clusters passed again. A class keeps them together and valid.

B3-L1 Dtypes in pandas 3, nullable and string

B3-L1 exercise 1

Hints. (1) Can NumPy's int64 hold NaN? (2) What does pandas use for NaN by default? (3) A float.

Answer. c) float64 Because: r1) NumPy int64 cannot hold a missing value, so pandas stores NaN in a float column.

NaN is a float, so a missing value upcasts integers to float64. Ask for Int64 to keep integers.

Common wrong answers.

- a: int64 has no way to represent a missing value.
- b: Int64 is used only when you ask for it, for example with `dtype="Int64"` or `convert_dtypes()`.
- d: Numbers with None infer a numeric dtype, not object.
- c|r2: Right answer, wrong reason. `pd.Series([4, 2])` is int64. The None forces the float.

B3-L1 exercise 2

Hints. (1) What dtype does `read_csv` infer for 01234? (2) What does the blank row add? (3) A number with a missing value is float64.

Answer.

```
1 1234.0
```

`read_csv` reads 01234 as a number and the blank as NaN, so the column is float64 and the leading zero is gone. Pass `dtype={"postcode": "str"}`.

Common wrong answers.

- 01234: Without a dtype map, read_csv infers a number. The blank row makes it float64.
- 1234: The blank row forces float64, so the value prints with .0.

B3-L1 exercise 3

Hints. (1) IDs are labels, not quantities. (2) Which integer dtype holds pd.NA? (3) str and Int64.

Answer.

```
1 df = pd.read_csv(path, dtype={"respondent_id": "str", "household_size":  
    "Int64"})
```

str keeps IDs as written. Int64 is the nullable integer dtype.

Common wrong answers.

- g2 = int64: Lower-case int64 is NumPy's integer and fails on a blank value. Use capital-I Int64.
- g1 = int: IDs are labels. Reading them as numbers drops leading zeros.

B3-L1 exercise 4

Hints. (1) read_csv takes a dtype mapping. (2) IDs as str, counts as Int64. (3) pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES).

Answer.

```
1 import io  
2  
3 import pandas as pd  
4  
5 PANEL_DTYPES: dict[str, str] = {"respondent_id": "str", "household_size":  
    "Int64"}  
6  
7  
8 def read_panel(text: str) -> pd.DataFrame:  
9     """Read panel CSV text with explicit dtypes.  
10  
11     Args:  
12         text: CSV content with respondent_id and household_size.  
13  
14     Returns:  
15         A frame with str IDs and nullable integer counts.  
16     """  
17     return pd.read_csv(io.StringIO(text), dtype=PANEL_DTYPES)
```

The dtype map fixes both columns at load time. In a real pipeline it comes from config, not code.

B3-L1 exercise 5

Hints. (1) What does `mean()` do with `<NA>`? (2) What does it do with 0? (3) Filling blanks with 0 adds fake answers.

Answer. The bug is on line 7.

Fixed version:

```
1 import pandas as pd
2
3
4 def mean_household(df: pd.DataFrame) -> float:
5     """Mean household size among respondents who answered."""
6     sizes = df["household_size"].astype("Int64")
7     return float(sizes.mean())
```

`mean()` skips `<NA>` by default. Filling with 0 turns non-answers into households of size zero and pulls the mean down.

Common wrong answers.

- Line 6: `Int64` is right: it keeps blanks as `<NA>`, which `mean()` skips. The problem is what happens next.
- Line 8: `mean()` is fine on its own. It skips `<NA>`, but by then the blanks are zeros.

B3-L2 Vectorise instead of apply

B3-L2 exercise 1

Hints. (1) Look for the option with no per-row Python code. (2) A vectorised call takes whole columns as arguments. (3) `np.where(cond, a, b)` picks from `a` or `b` element by element in compiled code.

Answer. c) `np.where(df["reach_flag"] == 1, df["final_weight"], 0.0)`

Because: r1) It evaluates the condition and the choice on whole arrays in compiled code.

`np.where` takes whole arrays and runs one compiled loop. The other three options call Python code once per row.

Common wrong answers.

- `a: apply(axis=1)` calls the lambda once per row in Python. It measured about 500 times slower than `np.where` on one million rows.
- `b: A list comprehension` beats `apply`, but it still runs a Python loop over every value.
- `d: iterrows()` builds a Series for each row. It is the slowest option here.

- `c|r2`: Right choice, wrong reason. Length has nothing to do with speed. `np.where` works on whole arrays in compiled code.
- `c|r3`: Right choice, wrong reason. There is no `lambda`, and `np.where` runs on one core. It is fast because it avoids a Python call per row.

B3-L2 exercise 2

Hints. (1) `None` in a float Series becomes `NaN`. (2) What does any comparison with `NaN` return? (3) Every comparison with `NaN` is `False`.

Answer.

```
1 [False, True, False]
```

`None` becomes `NaN` in a float64 Series, and `NaN > 1` is `False`. So the missing value is treated as "not above 1". This is the missing-value trap from the concept.

Common wrong answers.

- `[False, True, None]`: A comparison on a float Series returns plain booleans. `NaN > 1` evaluates to `False`, not to a missing value.
- `[False, True, nan]`: Comparisons do not propagate `NaN` on float64 data. They return `False`, which is why missing values get silently mislabelled.
- `[True, True, False]`: `0.0 > 1` is `False`. Only 1.5 exceeds 1.

B3-L2 exercise 3

Hints. (1) The function takes a condition, a value for True and a value for False. (2) Its name is the SQL keyword for filtering rows. (3) `np.where(cond, value_if_true, value_if_false)`, with zero as the false value.

Answer.

```
1 df["weighted_reach"] = np.where(df["reach_flag"] == 1,
    df["final_weight"], 0.0)
```

`np.where(cond, df["final_weight"], 0.0)` keeps the weight where the flag is 1 and returns 0.0 elsewhere, as one float array.

Common wrong answers.

- `g1 = select`: `np.select` takes a list of conditions and a list of choices. For one condition and two outcomes, use `np.where`.
- `g1 = if`: There is no `np.if`. The vectorised if-else in NumPy is `np.where`.
- `g2 = None`: `None` turns the result into an object array. Use `0.0` so the column stays numeric.

B3-L2 exercise 4

Hints. (1) Compute the total weight first, and return early when it is zero. (2) `df['final_weight'].where(df['reach_flag'] == 1, 0.0)` keeps weights for reached rows only. (3) Sum the masked weights, divide by the total, and wrap the result in `float()`.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4
5 def weighted_reach_pct(df: pd.DataFrame) -> float | None:
6     """Weighted share of respondents with reach_flag == 1.
7
8     Args:
9         df: Rows with reach_flag and final_weight.
10
11     Returns:
12         The weighted share, or None when the total weight is zero.
13     """
14     total = df["final_weight"].sum()
15     if total == 0:
16         return None
17     reached = df["final_weight"].where(df["reach_flag"] == 1, 0.0).sum()
18     return float(reached / total)
```

Sum the weights once for the denominator and return `None` when it is zero, which covers the empty frame too. Mask weights by the flag, sum, divide and convert to `float`.

B3-L2 exercise 5

Hints. (1) `pd.cut` turns numbers into labelled bands and keeps missing values missing. (2) Bins `[-np.inf, 35, 55, np.inf]` with `right=False` give `[.., 35)`, `[35, 55)`, `[55, ..)`. (3) Wrap it in `df.assign(age_group=pd.cut(...))` so you return a new frame.

Answer.

```
1 import numpy as np
2 import pandas as pd
3
4 AGE_BINS = [-np.inf, 35, 55, np.inf]
5 AGE_LABELS = ["16-34", "35-54", "55+"]
6
7
8 def add_age_group(df: pd.DataFrame) -> pd.DataFrame:
9     """Return a copy of df with an age_group column. Missing ages stay
10         missing."""
11     return df.assign(
```

```

11     age_group=pd.cut(df["age"], bins=AGE_BINS, labels=AGE_LABELS,
12                       right=False)

```

`pd.cut` with left-closed bins (`right=False`) matches the original `<` comparisons exactly. It keeps `NaN` as missing, which fixes a bug the original had. `assign` returns a new frame.

B3-L2 exercise 6

Hints. (1) What is `NaN >= 60`? (2) `np.where` has only two outcomes, so a missing value gets one of them. (3) Build the band as a Series, then keep it only where `minutes.notna()`.

Answer. The bug is on line 4.

Fixed version:

```

1 def add_viewing_band(df: pd.DataFrame) -> pd.DataFrame:
2     """Band weekly minutes into heavy (60 or more) and light. Missing
   minutes stay missing."""
3     minutes = df["minutes"]
4     band = pd.Series(np.where(minutes >= 60, "heavy", "light"),
5                      index=df.index)
6     return df.assign(viewing_band=band.where(minutes.notna()))

```

`NaN >= 60` is `False`, so `np.where` returns `"light"` for missing minutes. Wrap the result in a Series on the same index and apply `.where(minutes.notna())` to restore the missing values.

Common wrong answers.

- Line 3: Line 3 only selects the column. The missing value is lost where the condition is evaluated.
- Line 5: `assign` is right. It writes whatever `band` holds, and `band` is already wrong.

B3-L3 Method chaining and Copy-on-Write

B3-L3 exercise 1

Hints. (1) What does `df[mask]` return under Copy-on-Write? (2) Where does the second step write? (3) To a copy that is thrown away.

Answer. b) Nothing. pandas emits a `ChainedAssignmentError` warning. Because: r1) The first indexing step returns a new object, so the write lands on a temporary copy.

Two indexing steps mean two objects. The write hits the temporary one. Write in one step with `.loc`, or use `assign`.

Common wrong answers.

- `a`: Under Copy-on-Write the assignment goes to a temporary copy. Use `df.loc[mask, "weight"] = 0`.
- `c`: `ChainedAssignmentError` is a warning, not an exception. The code keeps running with `df` unchanged.
- `d`: The filter only selects rows. And the write never reaches `df` anyway.
- `b|r2`: Right answer, wrong reason. `df.loc[mask, col] = value` still changes `df`.

B3-L3 exercise 2

Hints. (1) Is `s` a view you can write through? (2) When does Copy-on-Write copy? (3) On the first write to shared data.

Answer.

```
1 [1, 2, 3]
```

`s` shares memory with `df` until you write to it. The write triggers a copy, so `df` keeps its values.

Common wrong answers.

- `[100, 2, 3]`: Under Copy-on-Write, `s` behaves as a separate object. Writing to it copies the data first, so `df` is unchanged.

B3-L3 exercise 3

Hints. (1) Which column names does each step need? (2) Which step depends on which rows are present? (3) Rename, filter, normalise, band.

Answer.

1. `.rename(columns={"wt": "weight"})`
2. `.loc[lambda d: d["age"] >= 18]`
3. `.assign(weight=lambda d: d["weight"] / d["weight"].mean())`
4. `.pipe(add_age_band, bins=BINS, labels=LABELS)`

Each step sees the output of the step above. Rename first so later steps use canonical names. A step that aggregates, such as a mean, must follow the filter that defines its population.

Common wrong answers.

- The normalising step reads the weight column, which does not exist until the rename.
- Normalising before the filter averages over minors too, so adult weights no longer average 1.0.

B3-L3 exercise 4

Hints. (1) Filter with `.loc[lambda d: ...]`. (2) Cap with `Series.clip(upper=cap)`. (3) Join them: `df.loc[...].assign(weight=lambda d: d["weight"].clip(upper=cap))`.

Answer.

```
1 import pandas as pd
2
3
4 def cap_weights(df: pd.DataFrame, cap: float) -> pd.DataFrame:
5     """Keep adults and cap their weights at cap."""
6     return df.loc[lambda d: d["age"] >= 18].assign(
7         weight=lambda d: d["weight"].clip(upper=cap)
8     )
```

`loc` then `assign` builds a new frame in two steps, with no temporary copies to write to.

B3-L3 exercise 5

Hints. (1) How many indexing steps does line 7 take before the `=`? (2) Where does the value land? (3) Use `.loc` in one step, or `Series.where` inside `assign`.

Answer. The bug is on line 7.

Fixed version:

```
1 import pandas as pd
2
3
4 def zero_invalid(df: pd.DataFrame) -> pd.DataFrame:
5     """Set weight to 0 where valid is False."""
6     return df.assign(weight=df["weight"].where(df["valid"], 0.0))
```

`out[mask]["weight"] = 0.0` writes to a temporary copy.

`df.assign(weight=df["weight"].where(df["valid"], 0.0))` keeps weights where valid and puts 0.0 elsewhere.

Common wrong answers.

- Line 6: `copy()` is harmless. Under Copy-on-Write it is also cheap until a write.
- Line 8: Returning `out` is fine. `out` never received the zeros.